МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №4

Специальность ПО-12

Выполнил

С. Э. Михновец,

студент группы ПО-12

Проверила

А. Д. Кулик,

преп. стаж. кафедры ИИТ,

«___k _____2026 г.

Брест 2026

1. Изложение цели работы

Цель работы: научиться работать с Github API, приобрести практические навыки написания программ для работы с REST API, извлечения связанных данных и их последующей визуализации с применением теории графов. Освоить стандарты оформления кода Python (Black, Pylint).

2. Задание по лабораторной работе (Вариант 5)

Построение карты взаимодействий разработчика на GitHub.
Используя GitHub REST API, необходимо написать Python-скрипт, который:

1. Запрашивает имя пользователя GitHub.
2. Получает список репозиториев (где пользователь делал коммиты, PR, issues, ставил звезды).
3. Находит других разработчиков, с которыми взаимодействовал пользователь.
4. Создает граф связей (узлы — пользователи, рёбра — взаимодействия).
5. Визуализирует граф (с помощью networkx и matplotlib) и сохраняет его в форматах PNG и JSON.

3. Спецификации ввода-вывода программы

- **Входные данные:**
  - Строка — имя пользователя (login) в GitHub, вводится через консоль (например, octocat).
- **Выходные данные (консоль):**
  - Информационные сообщения о статусе сбора данных.
  - Количество найденных связанных разработчиков.
- **Выходные файлы:**
  - github_network.json — данные графа в текстовом формате JSON (узлы и связи).
  - github_network.png — визуализация графа в виде изображения.

4. Текст программы

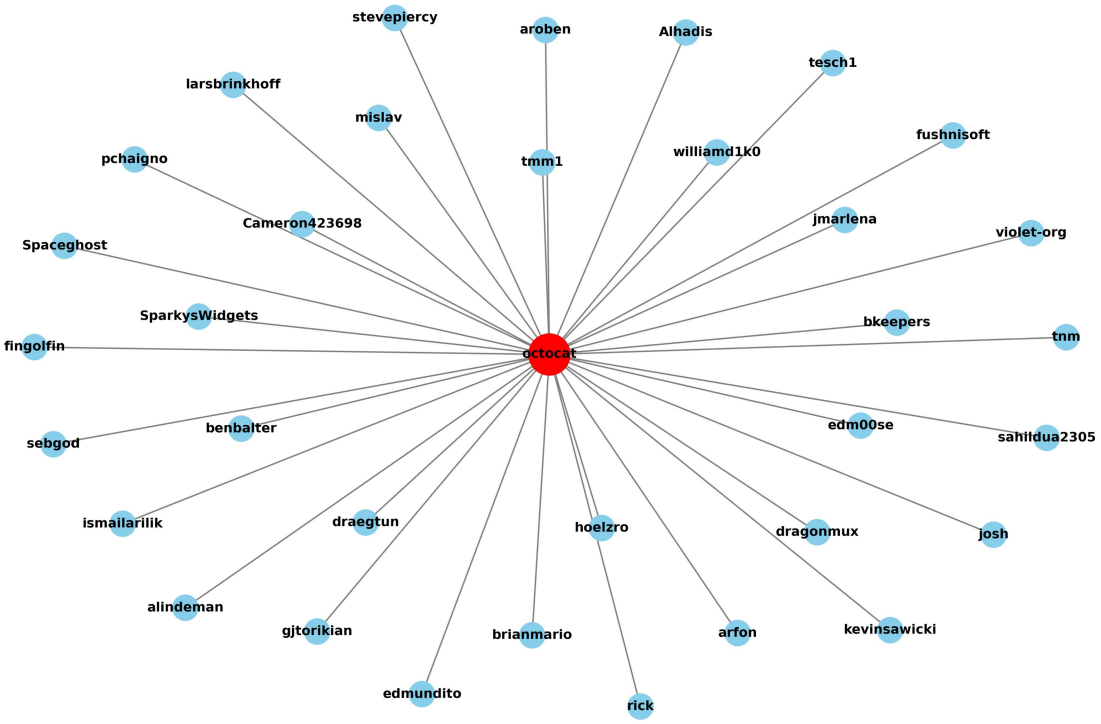
```
"""
Скрипт для построения графа взаимодействий разработчика на GitHub.
"""

import json
from typing import Dict, List, Set

import requests
import networkx as nx
import matplotlib.pyplot as plt

GITHUB_API_URL = "https://api.github.com"


def get_headers() -> Dict[str, str]:
    """Возвращает HTTP-заголовки для запросов к GitHub API."""
    return {"Accept": "application/vnd.github.v3+json"}


def fetch_starred_owners(username: str) -> List[str]:
```

```
    """Возвращает список владельцев репозиториев, отмеченных
пользователем."""
    url = f"{GITHUB_API_URL}/users/{username}/starred"
    response = requests.get(
        url, headers=get_headers(), params={"per_page": 30}, timeout=10
    )
    response.raise_for_status()
    return [repo["owner"]["login"] for repo in response.json()]


def fetch_event_interactions(username: str) -> List[str]:
    """Возвращает список пользователей из недавних событий."""
    url = f"{GITHUB_API_URL}/users/{username}/events"
    response = requests.get(
        url, headers=get_headers(), params={"per_page": 50}, timeout=10
    )
    response.raise_for_status()

    interacted = []
    for event in response.json():
        repo_data = event.get("repo")
        if not repo_data:
            continue
        repo_name = repo_data.get("name", "")
        if "/" in repo_name:
            owner = repo_name.split("/")[0]
            if owner != username:
                interacted.append(owner)
    return interacted


def fetch_repo_contributors(username: str) -> List[str]:
    """Возвращает список контрибьюторов репозиториев пользователя."""
    url = f"{GITHUB_API_URL}/users/{username}/repos"
    response = requests.get(
        url, headers=get_headers(), params={"per_page": 5}, timeout=10
    )
    response.raise_for_status()

    contributors = []
    for repo in response.json():
        contrib_url = repo.get("contributors_url")
        if not contrib_url:
            continue
        try:
            contrib_resp = requests.get(
                contrib_url, headers=get_headers(), timeout=10
            )
            if contrib_resp.status_code == 200:
                for user in contrib_resp.json():
                    if user["login"] != username:
                        contributors.append(user["login"])
        except requests.exceptions.RequestException:
            continue
    return contributors


def main() -> None:
    """Основная логика сбора данных и построения графа."""
    username = input("Введите имя пользователя GitHub: ").strip()
    if not username:
        print("Имя пользователя не может быть пустым.")
        return
```

```
    print(f"Анализируем взаимодействия пользователя {username}...")
    interactions: Set[str] = set()

    try:
        interactions.update(fetch_starred_owners(username))
        interactions.update(fetch_event_interactions(username))
        interactions.update(fetch_repo_contributors(username))
    except requests.exceptions.RequestException as error:
        print(f"Ошибка при обращении к API: {error}")
        return

    interactions.discard(username)
    print(f"Найдено {len(interactions)} связанных разработчика(ов).")

    graph = nx.Graph()
    graph.add_node(username, color="red", size=800)

    json_data = {"nodes": [{"id": username}], "links": []}

    for user in interactions:
        graph.add_node(user, color="skyblue", size=300)
        graph.add_edge(username, user)
        json_data["nodes"].append({"id": user})
        json_data["links"].append({"source": username, "target": user})

    with open("github_network.json", "w", encoding="utf-8") as file_out:
        json.dump(json_data, file_out, indent=4)
    print("Граф сохранён в github_network.json")

    plt.figure(figsize=(12, 8))
    pos = nx.spring_layout(graph, seed=42)

    colors = [graph.nodes[n].get("color", "skyblue") for n in graph.nodes]
    sizes = [graph.nodes[n].get("size", 300) for n in graph.nodes]

    nx.draw(
        graph,
        pos,
        with_labels=True,
        node_color=colors,
        node_size=sizes,
        font_size=9,
        font_weight="bold",
        edge_color="gray",
    )

    plt.title(f"Карта взаимодействий на GitHub: {username}")
    plt.savefig("github_network.png", format="png", dpi=300)
    print("Визуализация графа сохранена в github_network.png")


if __name__ == "__main__":
    main()
```

5. Выводы по проделанной работе

```
orts/Mihnovec_S_E/Lab_4/src/lab4.py
Введите имя пользователя GitHub: octocat
Анализируем взаимодействия пользователя octocat...
Найдено 34 связанных разработчика(ов).
Граф сохранён в github_network.json
Визуализация графа сохранена в github_network.png
PS D:\SPP>
```



Json:

```
{

    "nodes": [
        {
            "id": "octocat"
        },
        {
            "id": "tnm"
        },
        {
            "id": "alindeman"
        },
        {
            "id": "Spaceghost"
        },
        {
            "id": "jmarlena"
        },
```

```
        {
            "id": "larsbrinkhoff"
        },
        {
            "id": "josh"
        },
        {
            "id": "gjtorikian"
        },
        {
            "id": "benbalter"
        },
        {
            "id": "hoelzro"
        },
        {
            "id": "kevinsawicki"
        },
        {
            "id": "draegtun"
        },
        {
            "id": "williamd1k0"
        },
        {
            "id": "SparkysWidgets"
        },
        {
            "id": "rick"
        },
        {
            "id": "dragonmux"
        },
        {
            "id": "aroben"
        },
        {
            "id": "fushnisoft"
        },
        {
            "id": "edmundito"
        },
        {
            "id": "bkeepers"
        },
        {
            "id": "fingolfin"
        },
        {
            "id": "pchaigno"
        },
```

```
        {
            "id": "tmm1"
        },
        {
            "id": "edm00se"
        },
        {
            "id": "arfon"
        },
        {
            "id": "violet-org"
        },
        {
            "id": "tesch1"
        },
        {
            "id": "Alhadis"
        },
        {
            "id": "ismailarilik"
        },
        {
            "id": "sebgod"
        },
        {
            "id": "brianmario"
        },
        {
            "id": "sahildua2305"
        },
        {
            "id": "Cameron423698"
        },
        {
            "id": "mislav"
        },
        {
            "id": "stevepiercy"
        }
    ],
    "links": [
        {
            "source": "octocat",
            "target": "tnm"
        },
        {
            "source": "octocat",
            "target": "alindeman"
        },
        {
            "source": "octocat",
```

```
            "target": "Spaceghost"
        },
        {
            "source": "octocat",
            "target": "jmarlena"
        },
        {
            "source": "octocat",
            "target": "larsbrinkhoff"
        },
        {
            "source": "octocat",
            "target": "josh"
        },
        {
            "source": "octocat",
            "target": "gjtorikian"
        },
        {
            "source": "octocat",
            "target": "benbalter"
        },
        {
            "source": "octocat",
            "target": "hoelzro"
        },
        {
            "source": "octocat",
            "target": "kevinsawicki"
        },
        {
            "source": "octocat",
            "target": "draegtun"
        },
        {
            "source": "octocat",
            "target": "williamd1k0"
        },
        {
            "source": "octocat",
            "target": "SparkysWidgets"
        },
        {
            "source": "octocat",
            "target": "rick"
        },
        {
            "source": "octocat",
            "target": "dragonmux"
        },
        {
```

```
            "source": "octocat",
            "target": "aroben"
        },
        {
            "source": "octocat",
            "target": "fushnisoft"
        },
        {
            "source": "octocat",
            "target": "edmundito"
        },
        {
            "source": "octocat",
            "target": "bkeepers"
        },
        {
            "source": "octocat",
            "target": "fingolfin"
        },
        {
            "source": "octocat",
            "target": "pchaigno"
        },
        {
            "source": "octocat",
            "target": "tmm1"
        },
        {
            "source": "octocat",
            "target": "edm00se"
        },
        {
            "source": "octocat",
            "target": "arfon"
        },
        {
            "source": "octocat",
            "target": "violet-org"
        },
        {
            "source": "octocat",
            "target": "tesch1"
        },
        {
            "source": "octocat",
            "target": "Alhadis"
        },
        {
            "source": "octocat",
            "target": "ismailarilik"
        },
```

```
        {
            "source": "octocat",
            "target": "sebgod"
        },
        {
            "source": "octocat",
            "target": "brianmario"
        },
        {
            "source": "octocat",
            "target": "sahildua2305"
        },
        {
            "source": "octocat",
            "target": "Cameron423698"
        },
        {
            "source": "octocat",
            "target": "mislav"
        },
        {
            "source": "octocat",
            "target": "stevepiercy"
        }
    ]
}
```


В ходе выполнения лабораторной работы были получены следующие результаты:

1. Изучены принципы взаимодействия с GitHub REST API. Выполнена маршрутизация запросов к эндпоинтам /starred, /events и /repos с учетом ограничений базового лимита API.
2. Реализована обработка данных формата JSON для выявления логинов разработчиков, взаимодействующих с целевым пользователем